Tutorial de Simulação no Vivado

Projeto: Codificação Manchester Diferencial — Comunicação de Dados - Sistemas Reconfiguráveis — Prof. Dr. Vinícius S. Borges (2026/2)
Grupo 8: Arthur Benevides (082230016), Fernando Montanher (082230010), Guilherme Costa (081240041), Juan Haddad (081240043), Murillo Ando (082240042)

---

1. Pré-requisitos

- **Software:** AMD/Xilinx Vivado Design Suite (versão 2023.1, 2024.x ou 2025.2) com o **Vivado Simulator (XSim)** instalado.
- **Sistema Operacional:** Windows 10/11 (64 bits) ou Linux.
- **Repositório clonado localmente:** pasta **manchester_diferencial/** contendo **src/**, **sim/**, **constraints/** e **vivado_project/**.

2. Abertura do Projeto no Vivado

1. Inicie o **Vivado Design Suite**.
2. Na tela inicial (*Quick Start*), clique em **Open Project**.
3. Navegue até a pasta **manchester_diferencial/vivado_project/**, selecione o arquivo **manchester_diferencial.xpr** e clique em **OK**.
4. Alternativamente, caso deseje criar um projeto do zero: clique em **Create Project** → escolha *RTL Project* → adicione **src/manchester_dif.vhd** como *Design Source*, **sim/manchester_dif_tb.vhd** como *Simulation Source* e **constraints/manchester_dif.xdc** como *Constraints* → selecione a FPGA **xc7a35tcpg236-1**.

3. Estrutura dos Arquivos no Vivado

No painel **Sources** (aba *Hierarchy*), verifique a organização dos arquivos:

Categoria no Vivado	Arquivo	Papel no Projeto
Design Sources	`manchester_dif.vhd`	Módulo principal do codificador instanciado pelo testbench (UUT)
Design Sources	`manchester_dif_top.vhd` / `vga_pulse.vhd`	Módulos de topo e VGA utilizados na síntese para a placa Basys 3
Constraints (constrs_1)	`manchester_dif.xdc`	Mapeamento físico de pinos da Basys 3
Simulation Sources (sim_1)	`manchester_dif_tb.vhd`	Testbench auto-verificável (definido como Top Module da simulação)

4. Configuração da Simulação

1. Expanda **Simulation Sources** → **sim_1** e confirme que **manchester_dif_tb** está marcado com o ícone de **Top Module** (caso não esteja, clique com o botão direito sobre **manchester_dif_tb** e selecione **Set as Top**).
2. O testbench instancia o codificador com **CYCLES_PER_BIT = 10** e período de clock **CLK_PERIOD = 10 ns** (100 MHz), resultando em **BIT_PERIOD = 100 ns** (50 ns para cada metade de bit).
3. O tempo de simulação já está configurado como **-all** (o próprio processo de estímulo encerra o clock ao atribuir **sim_done <= true** por volta de **3.750 ns**).

5. Execução da Simulação Comportamental

1. No menu lateral esquerdo (**Flow Navigator**), localize a seção **SIMULATION**.
2. Clique em **Run Simulation** → **Run Behavioral Simulation**.

3. O Vivado compilará os pacotes VHDL, elaborará a entidade de teste e abrirá automaticamente a janela gráfica do **Waveform Viewer**.

6. Visualização e Leitura do Waveform

1. Na barra superior do Waveform Viewer, clique no botão **Zoom Fit** (ícone de quatro setas diagonais) para enquadrar toda a simulação de **0 ns a 4.500 ns**.
2. Verifique na coluna **Name** os sinais monitorados lado a lado: **clk**, **rst**, **data_in[15:0]**, **data_out**, **data_raw** e **led[15:0]**.
3. No console inferior (aba **Tcl Console**), confirme a mensagem: **Note: Testbench concluido.** sem nenhuma ocorrência de *Error* nas verificações automáticas por **assert**.

7. Resultado Esperado e Conferência das Formas de Onda

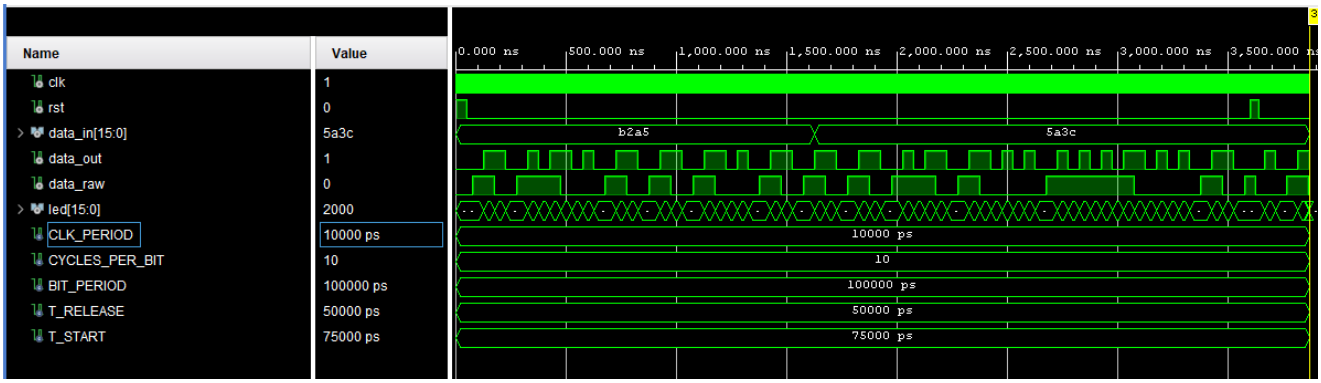

Figura 1 — Captura real da simulação comportamental de manchester_dif_tb no Vivado Simulator.

O testbench aplica três cenários consecutivos para validar todos os requisitos do enunciado:

- **1º Quadro (75 ns a 1.675 ns — DATA_A = 0xB2A5 = 1011_0010_1010_0101):** Após a liberação do reset em 50 ns e 2 ciclos do sincronizador, a transmissão inicia em 75 ns pelo bit **SW15 = '1'** até **SW0 = '1'**.
- **Troca Dinâmica e 2º Quadro (1.675 ns a 3.275 ns — DATA_B = 0x5A3C = 0101_1010_0011_1100):** Em 1.625 ns (meio do bit SW0 do 1º quadro), **data_in** muda para **0x5A3C**. O bit SW0 em andamento não é afetado (graças ao registrador **cur_bit**), e o novo valor entra em vigor no reinício automático em SW15 (1.675 ns).
- **Teste de Reset Assíncrono (3.400 ns):** Após transmitir os primeiros bits do 3º quadro, **rst** é acionado em '1', levando **data_out = '0'** e **led = 0x0000**; ao soltar o reset, a transmissão recomeça imediatamente em **SW15**.

Intervalo (ns)	Bit (Posição)	Valor (data_raw)	Nível Anterior	Início do Bit (1ª Met.)	Meio do Bit (2ª Met.)	Regra Verificada
`75 .. 175 ns`	SW15 (MSB)	`'1'`	`'0' (pós-reset)`	`'0' (mantém)`	`'1' (inverte)`	Bit 1 mantém nível pós-reset ('0')
`175 .. 275 ns`	SW14	`'0'`	`'1'`	`'0' (inverte)`	`'1' (inverte)`	Bit 0 inverte no início (1 → 0)
`275 .. 375 ns`	SW13	`'1'`	`'1'`	`'1' (mantém)`	`'0' (inverte)`	Bit 1 mantém no início (1 → 1)
`375 .. 475 ns`	SW12	`'1'`	`'0'`	`'0' (mantém)`	`'1' (inverte)`	Bits '1' consecutivos mantêm no início
`475 .. 575 ns`	SW11	`'0'`	`'1'`	`'0' (inverte)`	`'1' (inverte)`	Bit 0 inverte no início (1 → 0)
`575 .. 675 ns`	SW10	`'0'`	`'1'`	`'0' (inverte)`	`'1' (inverte)`	Bits '0' consecutivos invertem no início